

Apuntes sobre .env y la biblioteca phpdotenv

¿Qué es un archivo .env?

Un archivo .env es un **archivo de texto** que contiene pares clave-valor para almacenar configuraciones y variables sensibles de tu aplicación. Su objetivo principal es separar la configuración del código, permitiendo que las variables se definan de forma segura y que puedan cambiar según el entorno (desarrollo, pruebas, producción, etc.).

Ejemplo de un archivo .env:

```
ENV=desarrollo
DB_HOST=localhost
DB_PORT=3306
DB_DATABASE=mi_base_datos
DB_USERNAME=usuario
DB_PASSWORD=contraseña_secreta
```

Ventajas de usar un archivo .env

1. **Seguridad:** Permite mantener credenciales como contraseñas, claves API, y configuraciones sensibles fuera del código fuente.
2. **Flexibilidad:** Facilita cambiar configuraciones dependiendo del entorno sin necesidad de modificar el código.

¿Qué es vlucas/phpdotenv?

vlucas/phpdotenv es una biblioteca de PHP que permite cargar y gestionar las variables definidas en un archivo .env. Proporciona una forma sencilla de acceder a estas variables como si fueran parte del entorno del sistema.

Instalación de phpdotenv

Para usar esta biblioteca, necesitas instalarla con Composer.

Comando de instalación:

```
composer require vlucas/phpdotenv
```

Uso de phpdotenv

1. Cargar el archivo .env

Después de instalar la biblioteca, necesitas crear el archivo **.env** en la raíz de tu aplicación.

Ejemplo:

```
require_once __DIR__ . '../vendor/autoload.php';
use Dotenv\Dotenv;
$dotenv = Dotenv::createImmutable(__DIR__ . '../');
$dotenv->load();
```

2. Acceder a las variables de entorno

Una vez cargado el archivo **.env**, puedes acceder a las variables de dos maneras principales:

A través del array `$_ENV`:

```
// Usar $_ENV
```

```
echo $_ENV['DB_HOST']; // localhost
```

¿Por qué debemos usar .env y phpdotenv?

1. Seguridad

Al almacenar credenciales sensibles en un archivo **.env**, puedes excluir este archivo del control de versiones (como Git) mediante un archivo **.gitignore**. Así, las credenciales no se comparten accidentalmente.

2. Flexibilidad para múltiples entornos

Puedes crear un archivo **.env** diferente para cada entorno:

- **.env** para desarrollo.
- **.env.production** para producción.
- **.env.testing** para pruebas.

3. Mantenimiento del código

Es más fácil mantener configuraciones separadas del código fuente, haciendo que el código sea más limpio y fácil de leer.

Conclusión

El uso de un archivo `.env` junto con la biblioteca `vlucas/phpdotenv` proporciona un enfoque profesional, seguro y eficiente para gestionar configuraciones en aplicaciones PHP. Adoptar estas prácticas ayuda a desarrollar aplicaciones más flexibles y fáciles de mantener, reduciendo el riesgo de errores y aumentando la seguridad del proyecto.